SENTINEL-X

Unified Cyber Defense, Automated Containment & Cryptographic Digital Forensics Platform

Student / Architect: K. Jaideep Raj
Engineering Environment: Google Antigravity
Repository: github.com/jai07o/SENTINEL-X
Test Coverage: 95/95 Tests (100% Passed)

1. PROJECT TITLE & EXECUTIVE SUMMARY

SENTINEL-X: Real-Time Multi-Sector Endpoint Safeguard, Autonomous Intrusion Containment & Court-Admissible Digital Forensics Preservation

Core Purpose: Ordinary antiviruses simply delete files after detection, destroying legal proof and leaving systems vulnerable to zero-day in-memory attacks and offline hardware tampering. **SENTINEL-X** introduces the **Atomic Triad Architecture**: (1) *Real-Time Zero-Day Detection* across 6 physical sectors (Processes, Sockets, Hardware Cables, Wi-Fi, Privacy, OS Files); (2) *Volatile Scene Digital Forensics* capturing RAM state, open C2 sockets, and hashing evidence via SHA-256 and HMAC-SHA256 before modification; (3) *Autonomous Threat Containment* terminating rogue processes, severing connections, quarantining ports, and enforcing 24/7 zero-runtime OS hardening.

Key Innovations for Evaluation / Viva

- **Atomic Triad:** Detect → Cryptographically Seal → Contain.
- **Court-Admissible Evidence:** Volatile scene dump marked `0400` (read-only) with SHA-256 + HMAC-SHA256 digital signature.
- **Universal Cross-Device Profiler:** Dynamic detection of Windows (PC/Laptop), Linux (Server/Kali), macOS, Android (APK/Termux), and IoT (Raspberry Pi).
- **Crash-Proof Concurrency:** SQLite WAL mode (`PRAGMA journal_mode=WAL;`) with 30s busy timeout and 3-stage backoff.

Multi-Sector Defense Layers

Sector	Protection Focus
Hardware / USB	BadUSB / Rubber Ducky, Pendrive Viruses, Juice Jacking
Wireless / Wi-Fi	Evil Twin Rogue APs, Karma, ARP Spoofing MitM
Network Sockets	Inbound Backdoors (4444, 1337), Reverse C2 Shells
Host Processes	Cryptominers, Masquerading binaries, Dropper paths
Privacy Shield	Unauthorized background Webcam / Microphone snooping
Zero-Runtime	USB AutoRun disabled (0xFF), Storage Deny_Execute, 24/7 Watchdog

2. PROJECT FOLDER STRUCTURE

```
cyber_security/
├── backend/                            # Core High-Performance Backend (FastAPI + Async Python)
│   ├── app/
│   │   ├── api/v1/                     # REST & WebSocket Endpoints (ai.py, live_safeguards.py, etc.)
│   │   ├── core/                       # Config, Authentication (JWT), Middleware, Security Hardening
│   │   ├── models/                     # SQLAlchemy Database Schemas (Incident, EvidenceItem, Host)
│   │   ├── schemas/                    # Pydantic Request/Response Data Validation Models
│   │   └── services/                   # Core Business Logic & Multi-Sector Detection Engines
│   │       ├── ai_threat_investigator_service.py      # Real-Time 4-Signal Attack Correlation Engine
│   │       ├── containment_evidence_orchestrator.py   # Unified Containment & Forensic Sealing Engine
│   │       ├── digital_forensics_investigator_service.py # Volatile Forensics, Timelines & Evidence DB
│   │       ├── hardware_port_service.py               # USB/Cable Inspection & BadUSB Port Quarantine
│   │       ├── live_safeguard_rectifier.py            # Live Threat Hunter & Automated Rectifier
│   │       ├── persistent_shield_service.py           # 24/7 Zero-Runtime Deep OS Hardening & Registry
│   │       ├── total_360_safeguard_service.py         # 6-Sector Unified Perimeter Safeguard Orchestrator
│   │       └── wifi_guard_service.py                  # Rogue AP, Evil Twin & ARP MitM Defense
│   └── tests/                          # Automated Verification Test Suite (100% Pass Rate)
├── frontend/                           # Cyber Defense Tactical Web Dashboard (React + Vite)
├── desktop/                            # Native Desktop Application Shell (Electron / PyInstaller)
├── android/                            # Native Android Companion Mobile Application (Kotlin / Termux)
├── dist/                               # Production Binaries
│   └── android/                        # Compiled Standalone Mobile APK (sentinel-x-security.apk)
├── simulator/                          # Realistic Attack Simulation Pipeline (Safe Offline Evaluation)
├── scripts/                            # 24/7 OS-Native Scheduled Watchdog Daemons
│   └── persistent_guardian_watchdog.py   # Autonomous Background Perimeter Guard
├── data/                               # Central Isolated Forensic Evidence & Database Vault
│   ├── database/sentinel_x.db          # Central WAL Database (Unified investigation_evidence table)
│   ├── evidence/                       # Sealed Volatile Crime-Scene Snapshots (chmod 0400 Read-Only)
│   ├── forensics/                      # Process Memory Dumps & Chronological Timelines
│   ├── reports/                        # Signed Court-Admissible Rectification Certificates
│   └── quarantine/                     # Isolated Hostile Binaries and Quarantined Payloads
├── launcher.py                         # Unified Cross-Platform System Launcher & Mode Selector
└── cli.py                              # Full-Featured Interactive Command-Line Interface
```

Exam Architecture Tip: Emphasize the single central database `data/database/sentinel_x.db`. All signals from the AI investigator, digital forensics engine, hardware port guard, and network auditor converge into the unified `investigation_evidence` table.

3. MAIN CODE OF THE PROJECT (EXAM CODE SHEET)

File Reference: `backend/app/services/containment_evidence_orchestrator.py` and `ai_threat_investigator_service.py`.
Write this exact concise class in your exam. It demonstrates Universal Device Profiling, Volatile Crime-Scene Evidence Capture, SHA-256 + HMAC Sealing, Threat Neutralization, and Crash-Proof Database Logging.

```python
import os, sys, time, json, psutil, hashlib, hmac, sqlite3, platform
from datetime import datetime, timezone
from typing import Dict, Any

class SentinelXSecurityOrchestrator:
    """
    Core Unified Engine for SENTINEL-X:
    Evidence-Driven Endpoint Detection, Automated Containment & Forensics Preservation.
    Workflow: Detection -> Volatile Evidence Sealing -> Threat Containment -> DB Persistence.
    """
    HMAC_SECRET = b"SENTINEL_X_HMAC_COURT_SEAL_KEY_2026"
    DB_PATH = "data/database/sentinel_x.db"

    def get_host_device_info(self) -> Dict[str, str]:
        """Universal device profiler: identifies Windows, Linux, macOS, Android, or IoT."""
        hostname = platform.node() or "HOST_DEVICE"
        battery = psutil.sensors_battery()
        is_android = "ANDROID_ROOT" in os.environ or os.path.exists("/system/build.prop")

        if is_android:
            device_type, device_os = "Android Mobile", f"Android ({platform.release()})"
        elif sys.platform == "win32":
            device_type = "Windows Laptop" if battery else "Windows Desktop/Server"
            device_os = f"Windows {platform.release()}"
        else:
            device_type = "Linux Laptop" if battery else "Linux Workstation/Server"
            device_os = f"Linux ({platform.release()})"

        return {"hostname": hostname, "type": device_type, "os": device_os}

    def contain_and_preserve(self, threat_pid: int, threat_name: str, remote_c2: str, site: str) -> Dict[str, Any]:
        """Executes atomic containment and evidence preservation without destroying proof."""
        now = datetime.now(timezone.utc).isoformat()
        device = self.get_host_device_info()

        # PHASE 1: Capture Volatile Crime-Scene Memory & Process State
        try:
            proc = psutil.Process(threat_pid)
            volatile_dump = {
                "pid": threat_pid, "name": proc.name(), "cmdline": " ".join(proc.cmdline()),
                "exe": proc.exe(), "memory_bytes": proc.memory_info().rss,
                "active_sockets": [f"{c.raddr.ip}:{c.raddr.port}" for c in proc.net_connections() if c.raddr]
            }
        except Exception as pe:
            volatile_dump = {"pid": threat_pid, "status": f"Process handle unavailable: {pe}"}

        # PHASE 2: Cryptographic Sealing (SHA-256 Payload Hash + HMAC-SHA256 Signature)
        evidence_bytes = json.dumps(volatile_dump, sort_keys=True).encode("utf-8")
        sha256_hash = hashlib.sha256(evidence_bytes).hexdigest()
        hmac_signature = hmac.new(self.HMAC_SECRET, evidence_bytes, hashlib.sha256).hexdigest()

        # PHASE 3: Automated Threat Containment (Kill Rogue Process & Sever Sockets)
        try:
            target_proc = psutil.Process(threat_pid)
            target_proc.kill()
            target_proc.wait(timeout=2.0)
            containment_status = "NEUTRALIZED_AND_KILLED"
        except (psutil.NoSuchProcess, Exception) as ke:
            containment_status = f"DORMANT_OR_STOPPED ({ke})"

        # PHASE 4: Persist Evidence Dossier into Central Database (WAL Mode Concurrency)
        with sqlite3.connect(self.DB_PATH, timeout=30.0) as conn:
            conn.execute("PRAGMA journal_mode=WAL;")
            conn.execute("PRAGMA busy_timeout=30000;")
            conn.execute("""
                CREATE TABLE IF NOT EXISTS investigation_evidence (
                    id TEXT PRIMARY KEY, timestamp TEXT, from_where TEXT, site TEXT,
                    app TEXT, device_type TEXT, device_os TEXT, evidence_hash TEXT,
                    hmac_signature TEXT, details TEXT, status TEXT
                );
            """)
            conn.execute("""
                INSERT INTO investigation_evidence
                (id, timestamp, from_where, site, app, device_type, device_os, evidence_hash, hmac_signature, details, status)
                VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
            """, (
```

```
                f"EV-{int(time.time())}", now, remote_c2, site, threat_name,
                device["type"], device["os"], sha256_hash, hmac_signature,
                json.dumps(volatile_dump), containment_status
            ))
            conn.commit()

        return {"status": containment_status, "evidence_hash": sha256_hash, "device": device}
```

Why This Code Scores High in Exams

- **Evidence-Driven Remediation:** Extracts RAM metadata and network sockets *before* killing the process.
- **Legal Admissibility:** Dual cryptographic hashing (SHA-256 for integrity + HMAC-SHA256 for non-repudiation).
- **Crash-Proof Concurrency:** Uses SQLite WAL mode to prevent `database is locked` errors under concurrent threads.

Examination Viva Q&A Cheat Sheet

- **Q: Why not just delete the virus?**
  A: Deleting destroys evidence. SENTINEL-X captures volatile state to prove origin, attribution, and time of attack.
- **Q: How does it work on Android?**
  A: Built into `dist/android/sentinel-x-security.apk`, monitoring battery, background apps, and storage over Termux/Kotlin bindings.